OpenZeppelin

Training for Constraining

How to Prevent Circuit-Level Bugs

Levi Katyal Sledd and Vijay Singh

50-minute workshop



https://github.com/V-k-h/training-for-constraining

Outline

Motivation

Circuits and Constraints

Kinds of Underconstraint Bugs

Deep dive: gnark fake GLV

Finding underconstraints

Verification Infrastructure and AI

Wrap-up

Motivation: Underconstrained Circuits in 2026

System	Missing requirement	Reported consequence
Zcash Orchard	Binding of an ECC base-point cell	Counterfeit Orchard value
OpenVM pairing	Proper-subfield membership of a hint	Invalid pairing checks can pass
Scroll ecPairing	G2 subgroup membership	Sequencer/prover disagreement and finalization failure
ZKsync Airbender	Deferred constraints and verifier composition	Three criticals in the tooling assessment

Discovery does not establish prior exploitation. Detailed reports: appendix. Airbender: one critical circuit bug and two verifier-composition flaws.

Finite Fields

A field is a set of numbers in which you can add, subtract, multiply, and (divide except by zero).

Finite Fields

A field is a set of numbers in which you can add, subtract, multiply, and (divide except by zero).

Fields may be finite. Simplest example: prime fields $\mathbb{F}_p = \{0, \ldots, p-1\}$ with $+$ and $\cdot$ modulo p.

Finite Fields

A field is a set of numbers in which you can add, subtract, multiply, and (divide except by zero).

Fields may be finite. Simplest example: prime fields $\mathbb{F}_p = \{0, \ldots, p-1\}$ with $+$ and $\cdot$ modulo p.

Example: In $\mathbb{F}_{11}$, $3 \cdot 7 = 21 \mod 11 = 10$.

Finite Fields

A field is a set of numbers in which you can add, subtract, multiply, and (divide except by zero).

Fields may be finite. Simplest example: prime fields $\mathbb{F}_p = \{0, \ldots, p-1\}$ with $+$ and $\cdot$ modulo p.

Example: In $\mathbb{F}_{11}$, $3 \cdot 7 = 21 \mod 11 = 10$.

Note: can reduce at any time, allows for shortcuts such as $10 = -1$.

Finite Fields

A field is a set of numbers in which you can add, subtract, multiply, and (divide except by zero).

Fields may be finite. Simplest example: prime fields $\mathbb{F}_p = \{0, \ldots, p-1\}$ with $+$ and $\cdot$ modulo p.

Example: In $\mathbb{F}_{11}$, $3 \cdot 7 = 21 \mod 11 = 10$.

Note: can reduce at any time, allows for shortcuts such as $10 = -1$.

Inverse example: $3 \cdot 4 = 12 = 1 \mod 11$, so $4 = 3^{-1} = \frac{1}{3}$.

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

- w: private witness
- x: public witness
- Define $R(w, x)$ if $x = w^4 + 5$.

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

- w: private witness
- x: public witness
- Define $R(w, x)$ if $x = w^4 + 5$.

Relations and Witnesses

Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

Relations and Witnesses

Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

Applied example: $w = (w_0, \ldots, w_k)$ is a Merkle proof, $x = (x_0)$ is a root hash, and $R(w, x) : [w_0 \ldots w_k$ iteratively hash up to $x_0]$.

Arithmetic Circuits and Constraints

Arithmetic circuit: like a Boolean circuit, but

- wires carry values in $\mathbb{F}_p$.
- gates are $+$ and $\cdot$.

Arithmetic Circuits and Constraints

Arithmetic circuit: like a Boolean circuit, but

- wires carry values in $\mathbb{F}_p$.
- gates are $+$ and $\cdot$.

Arithmetic circuits $\equiv$ systems of *constraints*, one for each $+$ or $\cdot$ operation.

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here $x = (x_0)$ is public and $w = (w_0, w_1, w_2, w_3)$ is the entire private witness.

- w_1, w_2, w_3 are intermediate witness values; no separate witness category is needed.
- $(x, w) \vDash C$ means that all four constraints hold.

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

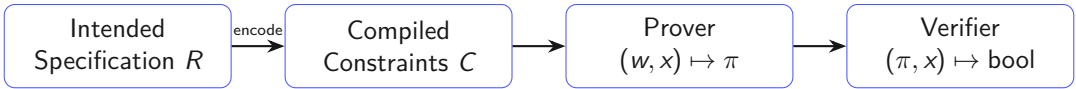
$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here $x = (x_0)$ is public and $w = (w_0, w_1, w_2, w_3)$ is the entire private witness.

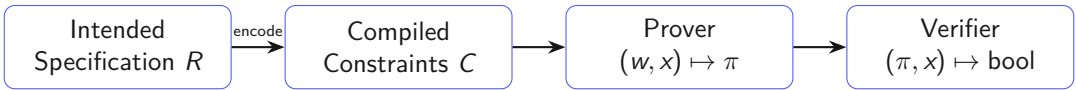
- w_1, w_2, w_3 are intermediate witness values; no separate witness category is needed.
- $(x, w) \vDash C$ means that all four constraints hold.

In this case, $(x, w) \vDash C$ implies $x_0 = w_0^4 + 5$. Conversely, when $x_0 = w_0^4 + 5$ we can choose intermediate values satisfying C.

A SNARK Only Proves the Encoded Relation



A SNARK Only Proves the Encoded Relation

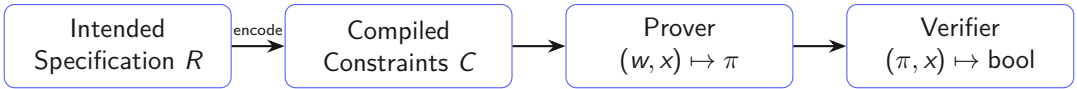


Ideally:

$$C(x, w) = 0 \quad \Longrightarrow \quad R(x, w)$$

For a public-statement specification: $\exists w : C(x, w) = 0 \Rightarrow \exists w : R(x, w)$.

A SNARK Only Proves the Encoded Relation



Ideally:

$$C(x, w) = 0 \quad \Longrightarrow \quad R(x, w)$$

For a public-statement specification: $\exists w : C(x, w) = 0 \Rightarrow \exists w : R(x, w)$.

An application can fail when this implication fails, even if the proof system works.

Hints

Honest computation

$$w_1 \leftarrow w_0^2$$

A hint or witness generator computes a value outside the circuit.

Enforced equation

$$w_1 - w_0^2 = 0$$

A constraint forces every accepted witness to respect that computation.

The adversarial model

The prover may replace the witness generator and every hint. Only the enforced constraints bind these choices.

Common Failure Patterns

Missing Requirement	What Can Go Wrong
Equality / copy constraint	Two wires meant to agree can differ.
Boolean or range constraint	A selector or amount takes unintended field values.
Hint validation	A prover controls the result of an expensive computation.
Field / group membership	Algebraic identities hold in a larger domain than intended.
Public-input binding	The proved computation no longer binds the claimed input.

Example: Missing Copy Constraint/Public Binding

Suppose we omit $C_3 : w_3 = x_0$ from our previous C.

Example: Missing Copy Constraint/Public Binding

Suppose we omit $C_3 : w_3 = x_0$ from our previous C.

Let $C' = C \setminus \{C_3\} = \{C_0, C_1, C_2\}$.

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

Example: Missing Copy Constraint/Public Binding

Suppose we omit $C_3 : w_3 = x_0$ from our previous C.

Let $C' = C \setminus \{C_3\} = \{C_0, C_1, C_2\}$.

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

For every x_0, the prover can pick any w_0, compute w_1, w_2, w_3, and get $(\overline{w}, x) \vDash C'$.

Example: Missing Binding $\rightarrow$ False Proof

Claim: "I know $w_0 \in \mathbb{F}_{11}$ such that $7 = w_0^4 + 5$", i.e. $w_0^4 = 2$.

Example: Missing Binding $\rightarrow$ False Proof

Claim: "I know $w_0 \in \mathbb{F}_{11}$ such that $7 = w_0^4 + 5$", i.e. $w_0^4 = 2$.

Fourth powers in $\mathbb{F}_{11}$:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

Note $R(w, 7)$ is false for every w.

Example: Missing Binding $\rightarrow$ False Proof

Claim: "I know $w_0 \in \mathbb{F}_{11}$ such that $7 = w_0^4 + 5$", i.e. $w_0^4 = 2$.

Fourth powers in $\mathbb{F}_{11}$:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

Note $R(w, 7)$ is false for every w.

But $\overline{w} = (1, 1, 1, 6)$ and $x_0 = 7$ satisfy C':

$$1 = 1^2, \qquad 1 = 1^2, \qquad 6 = 1 + 5.$$

Example: Missing Binding $\rightarrow$ False Proof

Claim: "I know $w_0 \in \mathbb{F}_{11}$ such that $7 = w_0^4 + 5$", i.e. $w_0^4 = 2$.

Fourth powers in $\mathbb{F}_{11}$:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

Note $R(w, 7)$ is false for every w.

But $\overline{w} = (1, 1, 1, 6)$ and $x_0 = 7$ satisfy C':

$$1 = 1^2, \qquad 1 = 1^2, \qquad 6 = 1 + 5.$$

So the prover can produce a valid proof of a false claim.

Repair: Public Input Binding

Add a copy constraint to bind an intermediate witness to the public witness

$$C_3 : x_0 = w_3.$$

Repair: Public Input Binding

Add a copy constraint to bind an intermediate witness to the public witness

$$C_3 : x_0 = w_3.$$

Now $(w, x) \vDash \{C_0, C_1, C_2, C_3\} \Leftrightarrow R(w, x)$.

Repair: Public Input Binding

Add a copy constraint to bind an intermediate witness to the public witness

$$C_3 : x_0 = w_3.$$

Now $(w, x) \vDash \{C_0, C_1, C_2, C_3\} \Leftrightarrow R(w, x)$.

This copy constraint is the only link between what the circuit computes and what the verifier sees.

Example: An Unvalidated IsZero Hint

Let $a \in \mathbb{F}_{11}$ be a **public input**, $z \in \mathbb{F}_{11}$ the **public output**; h is a private hint.

$$R((a, z), h): \quad z = \mathsf{IsZero}(a) = \begin{cases} 1 & a = 0, \\ 0 & a \neq 0. \end{cases}$$

A witness generator suggests $h = a^{-1}$ if $a \neq 0$, and $h = 0$ otherwise. The circuit enforces only

$$C_0: \qquad z = 1 - ah.$$

Example: An Unvalidated IsZero Hint

Let $a \in \mathbb{F}_{11}$ be a **public input**, $z \in \mathbb{F}_{11}$ the **public output**; h is a private hint.

$$R((a, z), h): \quad z = \mathsf{IsZero}(a) = \begin{cases} 1 & a = 0, \\ 0 & a \neq 0. \end{cases}$$

A witness generator suggests $h = a^{-1}$ if $a \neq 0$, and $h = 0$ otherwise. The circuit enforces only

$$C_0: \qquad z = 1 - ah.$$

Attack: take $a = 3$, $h = 0$, $z = 1$. Constraint C_0 holds, but the public statement $\mathsf{IsZero}(3) = 1$ is false.

Repair: Constrain the Hint's Effect

Add

$$\boxed{C_1 : \ az = 0.}$$

If $a \neq 0$, the field property forces $z = 0$, and C_0 then forces $h = a^{-1}$. If $a = 0$, C_0 forces $z = 1$; the value of h does not matter.

Repair: Constrain the Hint's Effect

Add

$$\boxed{C_1 : \ az = 0.}$$

If $a \neq 0$, the field property forces $z = 0$, and C_0 then forces $h = a^{-1}$. If $a = 0$, C_0 forces $z = 1$; the value of h does not matter.

Lesson

A hint need not have a unique representation. The *public behavior* must be pinned down by constraints.

Exercise: what does this circuit enforce?

Intended behavior: s selects x or y.

$$s \in \{0,1\}, \qquad z = \begin{cases} x & s = 1, \\ y & s = 0. \end{cases}$$

Implemented constraint over $\mathbb{F}_{11}$:

$$\boxed{z = sx + (1-s)y}$$

Fix public witnesss $x = 3$, $y = 7$, $z = 5$. The selector s is secret.

Two minutes with your neighbour

Can you choose s so the equation accepts? Which constraint is missing?

The accepted output can be neither input

For $s = 6$ in $\mathbb{F}_{11}$:

$$6 \cdot 3 + (1 - 6) \cdot 7 = 18 - 35 = -17 \equiv 5 \pmod{11}.$$

The implemented equation accepts, although $5 \notin \{3, 7\}$.

Repair

$$\boxed{s(s-1) = 0} \qquad \text{and} \qquad z = sx + (1 - s)y.$$

In a field, the new equation forces $s = 0$ or $s = 1$.

A Boolean check is a 1-bit range check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

A Boolean check is a 1-bit range check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

Field elements have no order and no sign. In $\mathbb{F}_{11}$, $3 - 5 = -2 = 9$.

A Boolean check is a 1-bit range check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

Field elements have no order and no sign. In $\mathbb{F}_{11}$, $3 - 5 = -2 = 9$.

So claims like "balance $\geq$ withdrawal" need a *range check*: reading x as an integer in $\{0, \ldots, p-1\}$, show $x < 2^n$.

Range checks by bit decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$

$C_1 : w_1(w_1 - 1) = 0$

$C_2 : x = w_0 + 2w_1$

Range checks by bit decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$

$C_1 : w_1(w_1 - 1) = 0$

$C_2 : x = w_0 + 2w_1$

Then $x \in \{0, 1, 2, 3\}$, so $(w, x) \vDash C \Leftrightarrow R(x) : x < 4$.

Range checks by bit decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$

$C_1 : w_1(w_1 - 1) = 0$

$C_2 : x = w_0 + 2w_1$

Then $x \in \{0, 1, 2, 3\}$, so $(w, x) \vDash C \Leftrightarrow R(x) : x < 4$.

Pitfall: this needs $2^n \leq p$. With 4 bits in $\mathbb{F}_{11}$, $x = w_0 + 2w_1 + 4w_2 + 8w_3$ and

$$3 = 1 + 2, \qquad 3 = 2 + 4 + 8 - 11.$$

Two bit patterns, $(1, 1, 0, 0)$ and $(0, 1, 1, 1)$, for the same x. If the circuit reads w_3 as "$x \geq 8$", the prover can claim $3 \geq 8$.

Arithmetic modulo a different number

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars). This is *foreign-field arithmetic*.

Arithmetic modulo a different number

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars). This is *foreign-field arithmetic*.

Toy: native field $\mathbb{F}_{11}$, foreign modulus $\ell = 3$. Public $x, y, z \in \{0, 1, 2\}$ (already range-checked).

Define $R(x, y, z)$ if $xy \equiv z \pmod 3$.

Arithmetic modulo a different number

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars). This is *foreign-field arithmetic*.

Toy: native field $\mathbb{F}_{11}$, foreign modulus $\ell = 3$. Public $x, y, z \in \{0, 1, 2\}$ (already range-checked).

Define $R(x, y, z)$ if $xy \equiv z \pmod 3$.

Over the integers, $xy \equiv z \pmod 3 \Leftrightarrow xy - z = 3k$ for some quotient k. The prover supplies k as a hint w:

$C_0 : xy - z = 3w$

$C_1 : w(w - 1) = 0$

Arithmetic modulo a different number

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars). This is *foreign-field arithmetic*.

Toy: native field $\mathbb{F}_{11}$, foreign modulus $\ell = 3$. Public $x, y, z \in \{0, 1, 2\}$ (already range-checked).

Define $R(x, y, z)$ if $xy \equiv z \pmod 3$.

Over the integers, $xy \equiv z \pmod 3 \Leftrightarrow xy - z = 3k$ for some quotient k. The prover supplies k as a hint w:

$C_0 : xy - z = 3w$

$C_1 : w(w - 1) = 0$

Example: $x = y = 2$, $z = 1$: $4 - 1 = 3 \cdot 1$, so $w = 1$.

Why a Boolean quotient suffices

$C_0 : xy - z = 3w$ holds in $\mathbb{F}_{11}$, i.e. $xy - z - 3w$ is a multiple of 11 as an integer.

Why a Boolean quotient suffices

$C_0 : xy - z = 3w$ holds in $\mathbb{F}_{11}$, i.e. $xy - z - 3w$ is a multiple of 11 as an integer.

With $x, y, z \in \{0, 1, 2\}$ and $w \in \{0, 1\}$:

$$xy - z - 3w \in \{-5, \ldots, 4\}.$$

Why a Boolean quotient suffices

$C_0 : xy - z = 3w$ holds in $\mathbb{F}_{11}$, i.e. $xy - z - 3w$ is a multiple of 11 as an integer.

With $x, y, z \in \{0, 1, 2\}$ and $w \in \{0, 1\}$:

$$xy - z - 3w \in \{-5, \ldots, 4\}.$$

The only multiple of 11 in that range is 0. So $xy - z = 3w$ holds over the integers, and $xy \equiv z \pmod 3$.

Why a Boolean quotient suffices

$C_0 : xy - z = 3w$ holds in $\mathbb{F}_{11}$, i.e. $xy - z - 3w$ is a multiple of 11 as an integer.

With $x, y, z \in \{0, 1, 2\}$ and $w \in \{0, 1\}$:

$$xy - z - 3w \in \{-5, \ldots, 4\}.$$

The only multiple of 11 in that range is 0. So $xy - z = 3w$ holds over the integers, and $xy \equiv z \pmod 3$.

The range checks keep every term small enough that the native field never wraps.

An unbounded quotient checks nothing

Suppose we omit $C_1 : w(w-1) = 0$.

An unbounded quotient checks nothing

Suppose we omit $C_1 : w(w-1) = 0$.

3 is invertible in $\mathbb{F}_{11}$ ($3^{-1} = 4$), so C_0 is satisfied by $w = (xy - z) \cdot 4$ for *any* x, y, z.

An unbounded quotient checks nothing

Suppose we omit $C_1 : w(w-1) = 0$.

3 is invertible in $\mathbb{F}_{11}$ ($3^{-1} = 4$), so C_0 is satisfied by $w = (xy - z) \cdot 4$ for *any* x, y, z.

False claim: $2 \cdot 2 \equiv 0 \pmod 3$. Take $w = (4-0) \cdot 4 = 16 = 5$:

$$3 \cdot 5 = 15 = 4 \text{ in } \mathbb{F}_{11}.$$

So $(w, x, y, z) = (5, 2, 2, 0) \vDash \{C_0\}$, but $R(2, 2, 0)$ is false.

An unbounded quotient checks nothing

Suppose we omit $C_1 : w(w-1) = 0$.

3 is invertible in $\mathbb{F}_{11}$ ($3^{-1} = 4$), so C_0 is satisfied by $w = (xy - z) \cdot 4$ for *any* x, y, z.

False claim: $2 \cdot 2 \equiv 0 \pmod 3$. Take $w = (4-0) \cdot 4 = 16 = 5$:

$$3 \cdot 5 = 15 = 4 \text{ in } \mathbb{F}_{11}.$$

So $(w, x, y, z) = (5, 2, 2, 0) \vDash \{C_0\}$, but $R(2, 2, 0)$ is false.

Over the integers, $4 - 0 = 3 \cdot 5$ is false. The field silently wrapped 15 around to 4.

An unbounded quotient checks nothing

Suppose we omit $C_1 : w(w-1) = 0$.

3 is invertible in $\mathbb{F}_{11}$ ($3^{-1} = 4$), so C_0 is satisfied by $w = (xy - z) \cdot 4$ for *any* x, y, z.

False claim: $2 \cdot 2 \equiv 0 \pmod 3$. Take $w = (4-0) \cdot 4 = 16 = 5$:

$$3 \cdot 5 = 15 = 4 \text{ in } \mathbb{F}_{11}.$$

So $(w, x, y, z) = (5, 2, 2, 0) \vDash \{C_0\}$, but $R(2, 2, 0)$ is false.

Over the integers, $4 - 0 = 3 \cdot 5$ is false. The field silently wrapped 15 around to 4.

The quotient *and* every other term must be range-checked, so that the native equation cannot wrap.

A Subgroup of Order 5 inside $\mathbb{F}_{11}^{\times}$

The multiplicative group $\mathbb{F}_{11}^{\times}$ has order 10. Its squares form a subgroup

$$H = \{1, 3, 4, 5, 9\}, \qquad |H| = 5.$$

A Subgroup of Order 5 inside $\mathbb{F}_{11}^{\times}$

The multiplicative group $\mathbb{F}_{11}^{\times}$ has order 10. Its squares form a subgroup

$$H = \{1, 3, 4, 5, 9\}, \qquad |H| = 5.$$

If $a, b \in H$ and $a^2 = b^2$, then $(ab^{-1})^2 = 1$. Since H has odd order, $ab^{-1} = 1$, so $a = b$.

Cancellation Needs the Subgroup

Outside H, cancellation can fail: $1^2 = (-1)^2$ although $1 \neq -1$ in $\mathbb{F}_{11}$.

A group-domain hypothesis is part of the theorem.

Cancellation Needs the Subgroup

Outside H, cancellation can fail: $1^2 = (-1)^2$ although $1 \neq -1$ in $\mathbb{F}_{11}$.

A group-domain hypothesis is part of the theorem.

For fake GLV, cancellation by $[v]$ is valid when the relevant point difference is in a subgroup of prime order ℓ and $v \not\equiv 0 \pmod{\ell}$.

Fake GLV: checking a hinted scalar multiplication

Goal Establish $Q = [s]P$ with fewer curve operations. The prover supplies short scalars u, v and the candidate point Q.

$$\underbrace{[u]P = [v]Q}_{\text{curve relation}}, \qquad \underbrace{u \equiv sv \pmod{\ell}}_{\text{scalar relation}}.$$

If P, Q belong to a prime-order subgroup of order ℓ and $v \not\equiv 0 \pmod{\ell}$, then

$$[v](Q - [s]P) = O \quad \Longrightarrow \quad Q = [s]P.$$

Every hypothesis needs enforcement.

Two fields meet in one gadget

Symbol	Meaning	Relevant arithmetic
r	Native circuit field modulus	Coordinates in $\mathbb{F}_r$
ℓ	Prime subgroup order	Scalars modulo ℓ

For BabyJubjub in a BN254 circuit, $r \neq \ell$.

$$u \equiv sv \pmod{\ell}$$ is a relation in the scalar domain.

An equation evaluated in $\mathbb{F}_r$ does not automatically enforce it.

A free quotient makes the check vacuous

A simplified form of the vulnerable binding check:

$$\boxed{u - sv = k\ell \quad \text{in } \mathbb{F}_r}$$

Here k is a prover-supplied field element without sufficient bounds. Since $\ell \neq 0$ in $\mathbb{F}_r$,

$$k = (u - sv)\ell^{-1} \pmod{r}$$

exists for *every* choice of u, s, v. The equation constrains k. It imposes no binding between u, s, and v.

The gnark assertion behind the algebra

```
// Excerpt from the supplied v0.15.0 case study
s, _ := api.NewHint(halfGCD, 4, checkedScalar, curve.Order)
s1, s2, bit, k := s[0], s[1], s[2], s[3]
vs := api.Mul(s2, checkedScalar)
kL := api.Mul(k, curve.Order)
lhs := api.Select(bit, s1, api.Add(s1, vs))
rhs := api.Select(bit, api.Add(kL, vs), kL)
api.AssertIsEqual(lhs, rhs)
```

For a Boolean sign bit, the branches assert $u - sv = k\ell$ or $u + sv = k\ell$ in $\mathbb{F}_r$. Both branches let the free quotient absorb the discrepancy. Bounds on s_1, s_2 do not bind k.

Source: Source excerpt: supplied HTML case study. PR #1765 is its cited repair.

What a false scalar multiplication means

Choose $u = 0$, $v = 1$, and $Q = O$.

$$[u]P = [v]Q = O.$$

The free quotient satisfies the native-field equation for any s. Thus the gadget can accept $Q = O$ even when $[s]P \neq O$.

Signature verification context

$$[S]B = R + [h]A, \qquad h = H(R, A, m).$$

If the gadget replaces $[h]A$ by O, the equation loses its intended key and message binding.

Fake GLV Repair: Restore Every Hypothesis

To infer $Q = [s]P$ from $[u]P = [v]Q$, require:

1. A **sound scalar congruence** $u \equiv sv \pmod{\ell}$ (proper foreign-field arithmetic or equivalent sound encoding).
2. P, Q in the intended **prime-order subgroup** of order ℓ.
3. $v \not\equiv 0 \pmod{\ell}$, so multiplication by v is invertible on that subgroup.
4. Sound constraints on signs, decomposition bounds, and the selected branches.

Then $[v](Q - [s]P) = O$ and cancellation gives $Q = [s]P$.

Crucial point

Bounding the quotient k *alone* does not establish the foreign-field relation. All terms need sound bounds/carry reasoning so native-field wraparound cannot invalidate the integer argument.

Source: gnark PR #1765 (verify precise affected paths and releases before attribution).

Honest-witness tests leave a gap

Typical positive test

$$w \leftarrow \mathsf{HonestWitness}(z), \qquad C(z, w) = 0.$$

Soundness question

$$\exists z, w : \; C(z, w) = 0 \;\wedge\; \neg R(z, w) \; ?$$

- Positive tests check examples of valid executions.
- Adversarial witness tests search outside the honest generator's outputs.
- Missing constraints can leave every positive test passing.

Manual review starts from the specification

1. Write the intended domain and input/output relation.
2. Mark every public witness, private wire, hint, and selector.
3. Trace the constraints that bind each required property.
4. Try a value outside the intended domain.
5. Add the missing constraint and retain the counterexample as a regression.

For the selector, the equation covered interpolation. The specification also required a Boolean domain.

Random Search versus Structured Witnesses

Randomly changing only public inputs often runs the honest witness generator and misses underconstraints.

Mutate private witnesses and hints: change a selector, quotient, or candidate point while keeping other constraints satisfiable.

Structured attacks beat blind random search: if a quotient k is free, calculate $k = (u - sv)\ell^{-1}$ directly rather than guessing it.

Demo: fuzz the relation, including the witness

```
# F_11; bypass the honest selector generator
x, y = 3, 7
for _ in range(1000):
    b = rng.randrange(11)
    z = (b*x + (1-b)*y) % 11
    if accepts(x, y, z, b) and z not in (x, y):
        report_counterexample(x, y, z, b)
        break
```

Oracle: acceptance plus a violation of the independent specification.

Randomize hints and private wires. Preserve the constraints you want to test.

Formal Verification: Search for an Invalid Witness

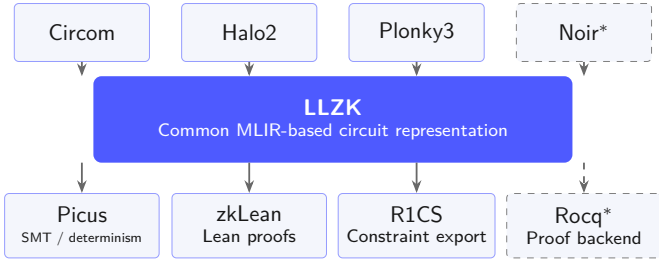
For public witness x and complete private witness w, check satisfiability of

$$\boxed{\exists x, w : \; C(x, w) = 0 \;\wedge\; \neg R(x, w).}$$

- **SAT:** a witness-level counterexample to the specification exists.
- **UNSAT:** no counterexample exists *in the encoded model*.
- **UNKNOWN / timeout:** no verification conclusion.

For the selector: $z = sx + (1 - s)y$ and $z \notin \{x, y\}$ is satisfiable. Add $s(s - 1) = 0$: the query becomes unsatisfiable over every field.

Formal Verification: Toward a Unified Framework



Why it matters

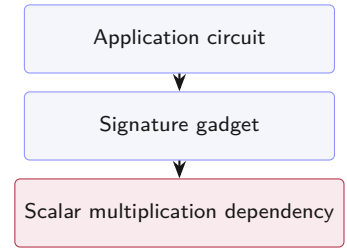
One intermediate representation enables reusable analyses across circuit languages; *LLZK is infrastructure, not itself a soundness proof*.

*Evolving integrations / research directions, not universal production support.

LLZK is open source, developed by Veridise with support from the Ethereum Foundation. https://github.com/project-llzk

Scalability note: Bit-blasting large-prime-field arithmetic into Boolean constraints can be costly. Generic SMT solvers may struggle with nonlinear field operations; specialized algebraic reasoning and circuit-aware verification backends help. LLZK provides the representation, not the solver.

Why AI-Assisted Review? The Dependency Problem



Why manual review alone is hard

- Security assumptions cross library and compiler boundaries.
- Advisories, versions, and reachable call paths change.
- Honest-witness tests can pass even when a dependency is underconstrained.

AI-assisted workflow

Find relevant advisories → **trace** affected call paths → **propose** malicious witnesses → **verify** using tests, solvers, or proofs.

AI broadens the search; executable evidence establishes the finding.

How We Review ZK Systems at OpenZeppelin

Auditor3 (AI-assisted)
Dependency tracing, hypotheses

MLIR / LLZK
Analyze circuit constraints

Lean proofs
Check mathematical formulas

Human auditors
Review assumptions and exploits

Complementary approaches

AI helps find places to look; formal methods establish selected properties; human reviewers connect the proofs and counterexamples to the real security model.

None alone establishes end-to-end security.

Questions

Which property does your circuit need to enforce?

Run the counterexamples yourself

```
git clone https://github.com/V-k-h/training-for-constraining
cd training-for-constraining
make demo # five counterexamples over F_11
make test # exhaustive checks of the repairs
make lean # optional: machine-checked theorems
```

Python 3.10+ only. Lean 4 and LLZK material is optional.



https://github.com/V-k-h/training-for-constraining

Appendix: source reports

- Shielded Labs: Orchard counterfeiting disclosure (June 4, 2026).
- Orchard missing-copy-constraint technical description.
- OpenVM: CVE-2026-46669 (May 15, 2026).
- Scroll Security Council: February 23 emergency upgrade report.
- Veridise: Airbender tooling report (April 22, 2026), pp. 3, 11–17.

Appendix: fake GLV and further reading

- Supplied reference: `fake-glv-underconstraint.html`. Main workshop uses its algebraic example, with explicit subgroup assumptions.
- gnark PR #1765: repair cited by the supplied reference.
- Fake GLV: original method discussion (2024).
- ZKSecurity: AI-assisted OpenVM discovery (2026).

The fake-GLV production version and patch claims come from the supplied reference. Check the release advisory before presenting exact affected-version ranges.

Appendix: a source-linked dependency review inventory

Published example	Review question
gnark signature malleability	Does the caller establish the required scalar range?
gnark range-check width	Does the enforced bound match the application's intended width?
gnark fake-GLV DoS	Can hostile inputs make witness generation stall? This is distinct from our soundness case.
OpenVM pairing soundness	Does the optimized check enforce the required subfield?

Illustrative inventory, not a complete issue list. Applicability requires version and call-path review.

Appendix: why short scalars help

$$u \equiv sv \pmod{\ell}, \qquad |u|, |v| \text{ roughly } \sqrt{\ell}.$$

The honest prover finds a short lattice vector, often with an extended-Euclidean or half-GCD method.

- Short scalars reduce the double-and-add work in $[u]P - [v]Q$.
- Finding the decomposition happens outside the circuit.
- The circuit still needs to enforce its algebraic relation, signs, bounds, and nonzero denominator.